The Islamia University of Bahawalpur

PROJECT PROPOSAL

AI Smart Price Comparison

An Intelligent Product Price Comparison System Using Machine Learning and NLP

A web application for smart product search, price comparison, price tracking, and buying decisions

Supervisor	Dr. Mustafa Hameed
Submitted by	Ali Muhammad Khan
Roll No.	F23BINFT1E02073
Submission Date	October 27, 2026

1 Introduction

Online shopping has made it easy to buy products from many stores, but the same product is often listed at different prices, with different delivery charges, stock availability, and product titles. Buyers usually open several websites one by one and compare listings manually, which is slow and error-prone.

This project proposes AI Smart Price Comparison, a web application that brings product listings from multiple permitted sources into one place. Machine Learning and Natural Language Processing (NLP) will be used to match listings that describe the same or an equivalent product,

even when their titles are written differently. The system will show the total estimated cost of each option, track price history, and notify users when a product reaches their target price.

The application will focus on transparent, reliable comparison. Each result will display its source, last-updated time, and match confidence so users can verify the information before making a purchase decision. The project will evaluate the product-matching component and the usefulness of the resulting web application.

2 Problem Statement

Product prices are scattered across many online stores. The same item may appear with different names, model formats, variants, and specifications, making manual comparison time-consuming. Looking only at the listed price is also misleading, because delivery fees, stock status, and product condition change the real value of an offer.

A simple keyword search does not solve this problem if it cannot reliably decide whether two listings refer to the same product. A web-based system is therefore needed that combines structured product data with an intelligent matching component, total-cost comparison, and price tracking, while clearly showing data freshness and uncertainty to the user.

3 Project Objectives

- To develop a web-based platform for searching and comparing products from multiple permitted store sources.
- To build a Machine Learning and NLP module that identifies exact and equivalent product listings across stores.
- To compare offers by total estimated cost, including price, delivery fee, and availability where data is available.
- To provide filters and sorting by price, brand, store, condition, and availability.
- To store price history and allow users to set target-price alerts for tracked products.
- To evaluate product matching using precision, recall, F1-score, and a confusion matrix.
- To assess usability and functional reliability through scenario-based testing and user feedback.

4 Proposed System

AI Smart Price Comparison will provide a browser-based platform where a user can search for a product by name, brand, model, or category. For example, a search such as “Samsung Galaxy A55 128GB” will return listings from supported stores; the system will normalize their titles, extract key attributes such as brand, model, storage, and color, and group matching listings together.

The Machine Learning component will compare listing pairs and classify them as an exact match, a similar item, or a different product, along with a confidence score. Listings with low confidence will be labelled “similar item” rather than presented as an exact match. Results will be ranked by total estimated cost, and each offer will show the store, price, delivery charges, stock status, last-updated time, and a link to the original listing.

Product data will be collected only from permitted APIs, approved data feeds, or a controlled sample dataset prepared for the project, in line with store terms and applicable laws. A price-history view will show how a product’s price changed over time, and users will be notified when a tracked product reaches their target price.

5 Main Modules

Module	Description
User Interface	Provides product search, comparison results, filters, product details, price-history charts, and alert settings.
Data Collection and Ingestion	Imports listings from permitted APIs, approved feeds, or the project dataset; validates and refreshes records on a schedule.
Product Normalization	Cleans product titles and extracts attributes such as brand, model, storage, size, color, and condition.
AI Product Matching Engine	Uses NLP features and a trained Machine Learning model to identify exact and equivalent listings with a confidence score.
Price Comparison Engine	Calculates total estimated cost and ranks offers by cost, relevance, and availability.
Price History and Alerts	Stores price snapshots, displays trends, and notifies users when a target price is reached.
Database and Authentication	Stores user accounts, products, listings, price snapshots, and alerts; protects user-specific data.
Evaluation and Administration	Supports test-set evaluation, error analysis, source management, and correction of incorrect matches.

6 Proposed Technologies

Programming Language	Python
Frontend	HTML, CSS, JavaScript
Backend	Flask

Machine Learning	Scikit-learn
NLP and Text Similarity	TF-IDF, fuzzy string matching, and optionally sentence embeddings (final choice subject to performance and hardware)
Database	SQLite initially; another relational database if required
Data Handling	Pandas and NumPy
Charts and Visualization	Chart.js for price-history graphs
Development Environment	Visual Studio Code
Version Control	Git and GitHub

7 Proposed Methodology

The project will follow an iterative development and evaluation process:

1. Collect requirements and define target users, supported product categories, data sources, and project boundaries.
2. Review relevant literature on price comparison systems, product matching, entity resolution, and text similarity.
3. Identify permitted data sources and prepare a sample dataset of product listings from selected categories.
4. Create a labeled dataset of listing pairs marked as exact match, similar item, or different product.
5. Clean and split the dataset into training, validation, and held-out test subsets; prevent near-duplicate examples from leaking across splits.
6. Implement title normalization and attribute extraction; train and compare suitable Scikit-learn matching models using validation performance and error analysis.
7. Design the database schema and implement authentication, search, comparison, price-history, and alert functions.
8. Build the web interface and integrate the matching and comparison engines with the Flask backend.
9. Test functional flows, data refresh, edge cases, and model performance; collect usability feedback from test users where permitted.
10. Document results, limitations, and future work; prepare the final demonstration and deployment.

8 Expected Outcomes

The expected outcome is a working web application that helps users find, compare, and track product prices from multiple sources in one place.

- A user-friendly platform for searching and comparing product listings.
- An AI-based product-matching model that identifies exact and equivalent listings within the defined scope.
- A comparison view that ranks offers by total estimated cost and availability.
- Price-history charts and a target-price alert workflow.
- A documented evaluation of matching performance, functional reliability, and usability.

Actual matching quality will be measured during testing and will depend on the dataset, the selected product categories, the available data sources, and the chosen model.

9 Project Scope

The initial version will focus on a limited number of product categories (such as mobile phones and laptops) and a small set of permitted data sources. It will include product search, AI-based listing matching, total-cost comparison, filters, price history, and price alerts. The project will demonstrate the feasibility of combining these functions in one application.

The system will not handle direct purchasing or checkout, guarantee coverage of every online store, or guarantee real-time prices. Mobile applications, additional stores and categories, and large-scale data collection may be considered future work rather than core requirements.

10 Limitations

- Matching performance will depend on the size, quality, and diversity of the labeled listing dataset.
- Coverage is limited to stores and sources that provide permitted access to their data.
- Prices and stock may change after the last refresh; results will show a last-updated time and stale-data warning.
- Some delivery fees, taxes, or discounts may be unavailable, so total cost will be labelled as an estimate.
- Products with very similar names or variants may be matched incorrectly; low-confidence matches will be flagged.
- The prototype may have limitations in scalability and store coverage compared with commercial comparison platforms.

11 Significance of the Project

AI Smart Price Comparison combines web development, Machine Learning, and NLP in a practical tool that saves time and money for online shoppers. It provides an opportunity to study how product-matching models perform on real-world, inconsistent listing data. The project also emphasizes transparency through source links, timestamps, and confidence labels instead of presenting every result as automatically correct.

12 Conclusion

The proposed AI Smart Price Comparison system aims to help users make better buying decisions by bringing product listings from multiple sources into one clear comparison. By combining Machine Learning-based product matching with total-cost comparison, price history, and alerts, the project will deliver a practical web application and an evaluable academic prototype. Its success will be assessed through model metrics, functional tests, and usability feedback, while keeping data sources permitted and the system’s limitations explicit.